

Three Engineering Execution Scenarios

Greenfield, brownfield, and ambiguous examples with decomposition, execution, and validation

Purpose

This document shows how the AI-assisted URL shortener can be planned, executed, and validated under three common engineering situations: building from scratch, enhancing an existing codebase, and resolving ambiguous requirements.

Project Context

- ? Java 21 Spring Boot 3.x modular monolith.
- ? APIs: POST /api/v1/urls, GET /{shortCode}, GET /api/v1/analytics/{shortCode}, GET /actuator/health.
- ? PostgreSQL stores durable URL and analytics data; Redis caches redirects.
- ? Docker Compose supports local runtime; Cloud Run deployment assets support production deployment.
- ? JUnit 5, Mockito, MockMvc, and Testcontainers cover verification.

Scenario 1 - Greenfield Build

Situation

A new service must be created from an HLD and functional requirements. The main risk is choosing a structure that is complete enough for production review without overcomplicating the first version.

Decomposition

Workstream	Output	Acceptance signal
Foundation	Gradle project, Java 21 toolchain, Spring Boot app, application.yml	Application context can start.
Domain model	UrlEntity and AnalyticsEntity mapped to PostgreSQL tables	Required URL and Analytics fields are represented.
Business services	URL creation, duplicate detection, code generation, redirect resolution, analytics capture	Service rules are unit tested.
HTTP API	Controllers for create, redirect, analytics, and health	Status codes and DTO payloads match requirements.
Operations	Dockerfile, docker-compose.yml, README, Swagger UI	App runs locally with PostgreSQL and Redis.
Testing	Unit, controller, repository, and integration tests	Fast and Docker-backed checks are available.

Execution

- ? Implemented package boundaries for controller, service, repository, cache, analytics, validation, mapper, DTO, entity, exception, config, and util.
- ? Created immutable DTO records and JPA entities for durable state.
- ? Used constructor injection and global exception handling.
- ? Added Redis read-through cache and PostgreSQL fallback.
- ? Captured analytics after successful redirects.

Validation

Validation type	Coverage
Unit tests	Creation, duplicate detection, invalid URL rejection, redirect behavior, and missing short code.
Controller tests	Create status codes, validation errors, redirect Location header, and analytics response.
Repository tests	PostgreSQL persistence and analytics ordering with Testcontainers.
Integration tests	Full create, redirect, analytics flow with PostgreSQL and Redis.
Manual smoke tests	Actuator health, Swagger UI, create API, redirect API, analytics API.

Scenario 2 - Brownfield Enhancement

Situation

An existing Spring Boot URL shortener already supports basic create and redirect behavior. The enhancement adds Redis cache, analytics persistence, Docker readiness, and production deployment files without destabilizing working APIs.

Decomposition

Existing area	Enhancement	Risk control
URL creation	Add duplicate handling and collision-safe generation.	Preserve endpoint behavior and add unit coverage.
Redirect	Add Redis lookup before PostgreSQL fallback.	Keep 302 redirect semantics unchanged.
Persistence	Add AnalyticsEntity and AnalyticsRepository.	Keep URL and analytics ownership separate.
API surface	Add GET /api/v1/analytics/{shortCode}.	Return 404 for unknown short codes.
Configuration	Add Redis, PostgreSQL, Actuator, Swagger, Docker Compose.	Use environment variables for deployment differences.
CI/CD	Add GitHub Actions and Cloud Run deployment assets.	Use Workload Identity Federation and Secret Manager.

Execution

- ? Introduced UrlCache so service code is not coupled to raw Redis operations.
- ? Added ClickAnalyticsService to separate click capture and analytics reads.
- ? Kept DTOs and mappers explicit to avoid leaking JPA entities.
- ? Split Docker-backed tests into integrationTest so gradle test remains fast.
- ? Added Cloud Run workflow that injects secrets at deploy time.

Validation

Gate	Expected result
Regression tests	Existing create and redirect behavior continues to pass.
Cache fallback test	Redirect works with Redis hit and PostgreSQL fallback.
Analytics test	Successful redirect increments persisted click analytics.
Repository test	Analytics rows are persisted and returned newest first.
Deployment verification	Cloud Run health endpoint returns UP after deploy.

Scenario 3 - Ambiguous Requirement

Situation

The prompt asks for a production-ready URL shortener but leaves open details such as authentication, custom domains, expiration, rate limits, migration tooling, analytics depth, and public access. The work is to make assumptions explicit and choose conservative defaults.

Decomposition

Ambiguity	Assumption	Implementation decision
Duplicate behavior	Same original URL should not create unlimited short codes.	Return existing short URL with 200 OK.
Short code format	Generated codes should be opaque and URL-safe.	Use random Base62 values.
Redirect performance	Redirect path should prefer low latency.	Use Redis read-through cache with PostgreSQL fallback.
Analytics detail	Capture useful but coarse request metadata.	Persist browser, device, referrer, timestamp.
Schema migrations	Assignment should be easy to run locally.	Use Hibernate ddl-auto update; document Flyway/Liquibase upgrade.
Deployment secrets	No secrets should be committed.	Use Secret Manager and GitHub OIDC.

Execution

- ? Documented key decisions and trade-offs.
- ? Avoided adding authentication, custom code support, expiration, malware scanning, or rate limiting because they were outside the explicit API contract.
- ? Used structured errors and clear status codes.
- ? Kept deployment configurable through environment variables and secret references.

? Preserved modular package separation for future features.

Validation

Validation question	How it is answered
Does the app compile and run?	Run gradle test and docker compose up --build.
Does it meet required APIs?	Smoke test create, redirect, analytics, and health endpoints.
Does Redis behavior work?	Run integrationTest with Docker Desktop.
Are secrets protected?	Inspect workflow and Cloud Run config for Secret Manager references.
Are trade-offs explicit?	Review architecture, deployment, and scenario PDFs.

Summary

Across all three scenarios, the delivery pattern is the same: decompose into bounded workstreams, execute in small verifiable increments, validate with automated and manual checks, and document assumptions when requirements leave room for interpretation.